

NATIONAL CYBER EMERGENCY RESPONSE TEAM OF PAKISTAN (PKCERT)

Government of Pakistan

MOBILE APPLICATION SECURITY ASSESSMENT REPORT

Document Version	1.0
Document Name	DIVA Android Application – Dynamic Security Analysis Report (Week 6)
Document No	PKCERT/INTERN-STT/2026/WK6/001
Classification	CONFIDENTIAL
Issuing Authority	National CERT of Pakistan (PKCERT) — Internship Program
Date of Issuance	30 August 2026

Important Note

This report reflects the security posture of the assessed application only at the time of assessment. Any subsequent modification, update, or rebuild of the application may affect its security posture and, therefore, may render the findings and conclusions of this report invalid. This assessment covers dynamic (runtime) analysis only; static analysis findings are documented separately in the Week 5 report and are referenced here where relevant.

This assessment was carried out as part of a supervised internship exercise on a deliberately vulnerable training application (DIVA).

Executive Summary

This report covers Week 6 of the Mobile Application Security track of the internship: dynamic (runtime) analysis of DIVA (Damn Insecure and Vulnerable App). Unlike Week 5, which examined the application without running it, Week 6 installed and ran the application on an Android Emulator, used Burp Suite to intercept all network traffic, and used ADB (Android Debug Bridge) to inspect the application's live files, SharedPreferences, SQLite databases, and system logs while it was in use.

Testing covered the application's Access Control, Insecure Data Storage, and Input Validation modules. Two Access Control screens were found to expose vendor API credentials with no authentication at all, simply by tapping a button. A separate Input Validation screen was found to be vulnerable to SQL Injection, where a single crafted search (' OR '1'='1) returned every user record in the database, including a credit card number, while a normal search correctly returned "user not found".

Dynamic testing also directly confirmed, with live evidence, three separate insecure-storage issues that had been predicted from source code in Week 5: credentials written in plain text to a file on external storage, a password and PIN stored in plain text in the app's SharedPreferences, and a second set of plaintext credentials stored in a local SQLite database. Throughout testing, Burp Suite confirmed that the application makes no external network calls at any point — all of its security weaknesses are in how it handles data and access control on the device itself.

In total, 9 dynamic findings were recorded: 3 Critical, 3 High, 1 Medium, and 2 Informational. A full breakdown of each finding, the steps used to reproduce it, proof-of-concept evidence, and remediation recommendations are provided in the following sections.

1. Summary of Findings

1.1 Basic Information

Target Owner	PKCERT — Internal Training Lab
Target Custodian	Hafiz Muhammad Moawiz (Trainee, Software Security Testing)
Target Name	DIVA — Damn Insecure and Vulnerable App (DivaApplication.apk)
Target Type	Android Application (APK) — Dynamic (Runtime) Analysis
Package Name	jakhar.aseem.diva
Target IP / URL	N/A — no external network endpoint identified during testing
Assessment Type	Mobile Application Security Assessment — Dynamic Analysis (Emulator + Network Interception + Runtime Storage Inspection)
Assessment Date	30 August 2026
Tools Used	Android Emulator (Pixel 8, Android 14 / API 34), Burp Suite Community Edition, Android Debug Bridge (ADB), Python sqlite3 module

1.2 Consolidated Vulnerability Summary

Severity	Count	% of Total
Critical	3	33.3%
High	3	33.3%
Medium	1	11.1%
Low / Informational	2	22.3%
TOTAL	9	100%

1.3 Consolidated Risk Overview

The overall risk rating for this application, based on dynamic testing, is Critical. Two Access Control screens (Findings D1 and D2) were found to expose vendor API credentials with no authentication step at all — simply tapping a button on either screen reveals live API keys, usernames, and passwords. A separate screen (Finding D3) was found to be vulnerable to SQL Injection, where a single crafted search query returns every user record in the database, including a credit card number, while normal searches behave correctly.

Dynamic testing also confirmed — with live, reproducible evidence — that the application stores authentication data in plain text across three separate mechanisms: an external file, SharedPreferences, and a local SQLite database. All of these were readable directly from the device's file system using only standard ADB commands, without needing to modify or reverse-engineer the application.

Testing with Burp Suite confirmed that the application makes no external network calls at any point during normal use. This means the “insecure API calls” and “improper SSL pinning” checks required by this week's task are not applicable in the traditional network sense: there is no API traffic to intercept and no TLS channel for pinning to protect. The application's risk is entirely in on-device access control and data storage, not in network communication.

1.4 Testing Environment

The table below summarizes the environment and tools used for this week's dynamic testing.

Attribute	Value
File Name	DivaApplication.apk
Package Name	jakhar.aseem.diva
Test Device	Android Emulator — Pixel 8, Android 14 (API 34), x86_64
Proxy Configuration	Burp Suite Community Edition, WiFi manual proxy 10.0.2.2:8080, CA certificate installed on emulator
Network Traffic Observed	None from the DIVA package — application operates entirely offline
Modules Tested	Access Control (Parts 1–2), Insecure Data Storage (Parts 1–4), Input Validation (Part 1)

2. Vulnerability Assessment

Dynamic testing was carried out by installing DIVA on an Android Emulator (Pixel 8, Android 14) and interacting with each test module as a normal user would, while three observation channels were kept active throughout: Burp Suite (to capture any network traffic), Android logcat (to observe system-level activity such as intent launches), and ADB shell access (to inspect files, SharedPreferences, and SQLite databases created by the app at runtime).

Several findings below directly confirm, with live evidence, predictions that were made from static source-code analysis in the separate Week 5 report; these are noted where relevant.

2.0 Findings Summary

#	Finding	Severity	Status
D1	Access Control Bypass — Vendor API Credentials Exposed (AccessControl1 → APICredsActivity)	Critical	Confirmed
D2	Access Control Bypass — "Already Registered" Skips PIN Verification (AccessControl2 → APICreds2Activity)	Critical	Confirmed
D3	SQL Injection — Single Payload Discloses All User Data Including Credit Card	Critical	Confirmed
D4	Credentials Written in Plain Text to External Storage (.uinfo.txt)	High	Confirmed
D5	Plaintext SharedPreferences (Username, Password, and PIN)	High	Confirmed
D6	Plaintext Credentials in a Second SQLite Database (ids2 / myuser table)	High	Confirmed
D7	Private Notes Stored Unencrypted in SQLite (divanotes.db)	Medium	Confirmed
D8	AccessControl3NotesActivity — Not Externally Exported, PIN Enforced	Informational	Confirmed Safe
D9	No External Network Communication Observed — SSL Pinning Not Applicable	Informational	Confirmed

Detailed steps, results, and evidence for each finding are given below.

2.1 Access Control Bypass — Vendor API Credentials Exposed

Severity: Critical **Status:** Confirmed

What we did:

The “Access Control Issues - Part 1” screen was opened and its available option was used as a normal user would. Android’s system log (logcat) was monitored throughout, and Burp Suite was active to capture any network traffic.

- Opened the “9. ACCESS CONTROL ISSUES - PART 1” screen and interacted with its credential-view option.

- The app immediately navigated to a “Vendor API Credentials” screen with no PIN, password, or any confirmation step.
- Checked logcat at the same moment to confirm what triggered the screen change.

What we found:

The credentials screen displayed: API Key: 123secretapikey123, API User name: diva, API Password: p@ssword. logcat confirmed the exact trigger: an Android intent with a custom action (jakhar.aseem.diva.action.VIEW_CREDS) launched APICredsActivity directly, with no authentication check anywhere in the flow.

Impact:

Any user, or any other app on the device that can send this intent, can retrieve live vendor API credentials with zero authentication. This is a pure application-layer access control failure that requires no network access, password, or special device privilege.

Evidence: Figures 4 and 5

2.2 Access Control Bypass — “Already Registered” Skips PIN Verification

Severity: Critical **Status:** Confirmed

What we did:

The “Access Control Issues - Part 2” screen was opened. This screen offers two options: “Register Now” (which correctly requires a PIN received from the simulated vendor) and “Already Registered” (intended for users who have registered previously).

- Selected “Already Registered” without ever completing the “Register Now” / PIN flow.
- Tapped “VIEW TVEETER API CREDENTIALS”.
- Observed the result on screen and cross-checked logcat for the triggering event.

What we found:

The credentials were displayed immediately, with no PIN requested at any point: TVEETER API Key: secrettveeterapikey, API User name: diva2, API Password: p@ssword2. logcat confirmed a second custom intent action (jakhar.aseem.diva.action.VIEW_CREDS2) launched APICreds2Activity directly. This is a business logic flaw: the app trusts the “Already Registered” choice without verifying it against any stored proof of prior registration.

Impact:

This is functionally identical in impact to Finding D1: a second, independent path to the same class of credential exposure, showing this is a systemic access control weakness rather than an isolated bug in one screen.

Evidence: Figures 6, 7 and 8

2.3 SQL Injection — Single Payload Discloses All User Data Including Credit Card

Severity: Critical **Status:** Confirmed

What we did:

The “Input Validation Issues - Part 1” screen was tested. Its stated objective is to retrieve all user records with a single malicious search, without knowing any valid username.

- Entered a normal, non-existent username into the search field and searched — the app correctly returned “user not found”.

- Entered the payload: ' OR '1'='1 into the same search field and searched again.

What we found:

The injected query returned every record in the users table in a single response: User: admin, pass: passwd123, Credit card: 1234567812345678 (further records were also present). The contrast with the earlier “user not found” result confirms the search field does not sanitise input before using it inside a SQL query, allowing the query's logic to be overridden.

Impact:

This is the most severe finding in this assessment: a single crafted string, entered through a normal search box with no special tools, discloses admin credentials and a full credit card number. In a real application this would allow complete compromise of the user database from the client side alone.

Evidence: Figure 9

2.4 Credentials Written in Plain Text to External Storage (.uinfo.txt)

Severity: High **Status:** Confirmed

What we did:

Test credentials were entered into the “Insecure Data Storage - Part 2” screen (3rd Party Service Username / Password fields), and the device's external storage was then inspected using ADB.

- Entered username testuser and password testpass123, then tapped Save.
- The app initially returned a “file error” — resolved by granting WRITE_EXTERNAL_STORAGE and READ_EXTERNAL_STORAGE permissions manually via adb shell pm grant, since the emulator runs a much newer Android version than the app was designed for.
- Re-entered the credentials and saved again — the app reported “3rd party credential saved successfully”.
- Ran adb shell, then cd /sdcard, ls -la, and cat .uinfo.txt.

What we found:

The file .uinfo.txt was present directly under /sdcard (a shared, non-app-private location) and contained: testuser:testpass123 in plain text.

Impact:

Any other application on the device with storage read permission, or anyone with physical/ADB access to the device, can read these credentials directly from the file system, with no need to reverse-engineer or exploit the app itself.

Evidence: Figure 10

2.5 Plaintext SharedPreferences (Username, Password, and PIN)

Severity: High **Status:** Confirmed

What we did:

The app's private SharedPreferences file was inspected using ADB's run-as capability, which is available because the app is a debuggable build.

- Ran adb shell, then run-as jakhar.aseem.diva, then cd shared_prefs and ls.
- Displayed the single preferences file found: cat jakhar.aseem.diva_preferences.xml.

What we found:

The file contained three plaintext string values: password (testuser), user (testuser), and notespin (1234). The notespin value is the PIN used to protect the Private Notes feature (Access Control - Part 3), confirming that this PIN is stored as plain text with no hashing or encryption.

Impact:

Any user with ADB access to the device — even without root — can read this file directly and recover the stored password and PIN in plain text.

Evidence: Figure 11

2.6 Plaintext Credentials in a Second SQLite Database (ids2 / myuser table)

Severity: High **Status: Confirmed**

What we did:

While investigating the app's local databases, two SQLite files were found under the app's private data directory. Since sqlite3 was not available inside the emulator shell, the database files were copied to the host machine using `adb exec-out ... run-as ... cat`, and queried locally using Python's built-in sqlite3 module.

- Copied both files: `adb exec-out run-as jakhar.aseem.diva cat databases/divanotes.db`, and the same for `databases/ids2`.
- Listed tables in `ids2.db`: `SELECT name FROM sqlite_master` — returned `android_metadata` and `myuser`.
- Queried the table: `SELECT * FROM myuser`.

What we found:

The `myuser` table returned two rows of plaintext username/password pairs entered during testing: ('testt', 'testtt') and ('test', 'testt'). No hashing, salting, or encryption was applied to either column.

Impact:

This is a second, independent instance of the same insecure-storage pattern seen in Findings D4 and D5 — the application consistently stores authentication data in plain text across multiple storage mechanisms (files, SharedPreferences, and SQLite).

Evidence: Figures 12 and 13

2.7 Private Notes Stored Unencrypted in SQLite (divanotes.db)

Severity: Medium **Status: Confirmed**

What we did:

The same `divanotes.db` file copied in Finding D6 was inspected for the “Private Notes” feature reached through the Access Control - Part 3 PIN screen.

- Listed tables: `SELECT name FROM sqlite_master` — returned `android_metadata`, `notes`, and `sqlite_sequence`.
- Queried the notes table: `SELECT * FROM notes`.

What we found:

All six private notes shown on-screen after entering the correct PIN were found stored in plain, readable text in the notes table (e.g. office → “10 Meetings. 5 Calls. Lunch with CEO”, home → “Buy toys for baby, Order dinner”, and others covering exercise, expense, holiday, and weekend plans).

Impact:

Although this data is only shown in the UI after a correct PIN is entered, the underlying storage has no encryption at rest, so anyone with file-level access to the app's data directory (enabled by the debuggable build) can bypass the PIN screen entirely and read this data directly.

Evidence: Figures 12, 13 and 14

2.8 AccessControl3NotesActivity — Not Externally Exported, PIN Enforced (Confirmed Safe)

Severity: Informational **Status:** Confirmed Safe

What we did:

To check whether the PIN screen protecting Private Notes could be bypassed the same way Findings D1 and D2 were (by launching the destination activity directly), an attempt was made to start AccessControl3NotesActivity directly via ADB, bypassing the PIN entry screen entirely.

- Ran: `adb shell am start -n jakhar.aseem.diva/.AccessControl3NotesActivity`

What we found:

The command failed with a `SecurityException`: “not exported from uid 10190”. Unlike `APICredsActivity` and `APICreds2Activity`, this component has no intent-filter and is correctly not exported, so it cannot be launched directly from outside the app. When the same screen was reached through the normal in-app flow, the PIN was still correctly requested and enforced.

Impact:

This positive result shows that access control in this application is inconsistently implemented: some sensitive screens (D1, D2) are exposed with no protection at all, while others (D8) are correctly locked down. This is useful for prioritising remediation, since it shows the developers know how to implement this control correctly elsewhere in the same app.

Evidence: Command output (terminal-only test, no image)

2.9 No External Network Communication Observed — SSL Pinning Not Applicable

Severity: Informational **Status:** Confirmed

What we did:

Burp Suite Community Edition was configured as a proxy for the Android Emulator (via the emulator's WiFi manual proxy settings, pointing to 10.0.2.2:8080), and Burp's CA certificate was installed as a trusted certificate on the emulator to allow HTTPS interception. This setup was verified as working using the emulator's browser before testing DIVA.

- Confirmed Burp was correctly intercepting traffic by browsing to an external HTTPS site in the emulator's browser and observing decrypted requests in Burp's HTTP History.
- Kept Burp's Intercept and HTTP History active throughout all Week 6 testing of every DIVA screen (Access Control, Insecure Data Storage, Input Validation).

What we found:

Across the entire testing session, the only traffic observed in Burp originated from Android system/Chrome components (e.g. a background request to `update.googleapis.com` for Chrome component updates) — unrelated to DIVA. No request from the `jakhar.aseem.diva` package was observed at any point.

Impact:

Since the app makes no network calls, the “insecure API calls” and “improper SSL pinning” checks required by this week's task do not apply to this application in the traditional sense — there is no API traffic to be

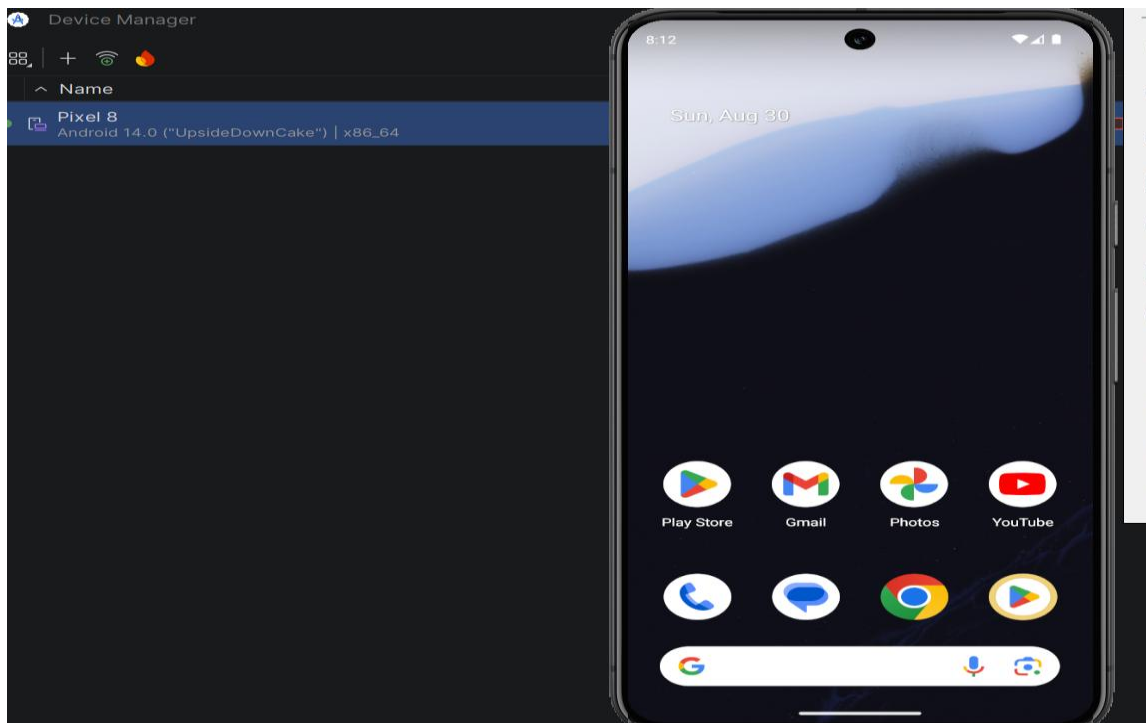
insecure, and no TLS channel for pinning to protect. This was still an important test to perform, since it rules out network-layer risk with evidence rather than assumption, and shows that DIVA's security weaknesses are entirely in on-device data handling and access control rather than in network communication.

Evidence: Figures 2 and 3 (Burp setup and interception confirmation)

3. Proof-of-Concept Evidence

All screenshots below were captured live during Week 6 dynamic testing and correspond directly to the findings described in Section 2.

Figure 1 — Test Environment



Android Emulator (Pixel 8, Android 14, API 34) running with DIVA installed and ready for testing.

Figures 2–3 — Burp Suite Interception Setup and DIVA Test Modules

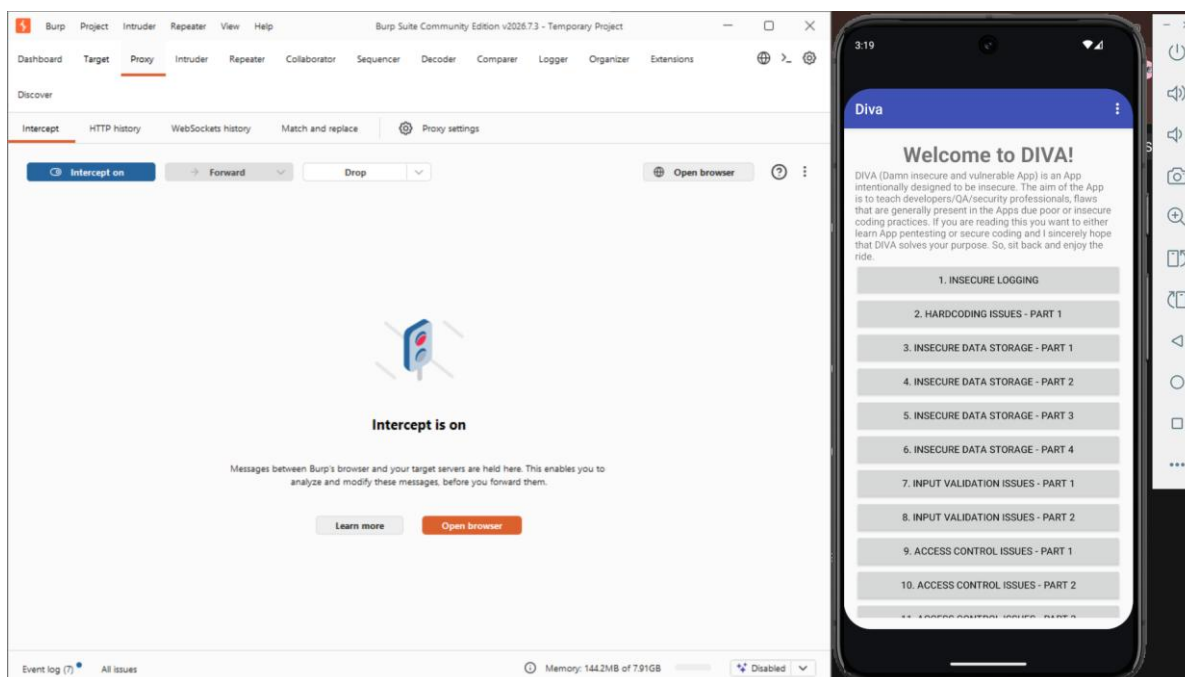


Figure 2 — Burp Suite intercepting live HTTPS traffic from the emulator's browser, confirming the proxy and CA certificate setup is functional, alongside the DIVA home screen.

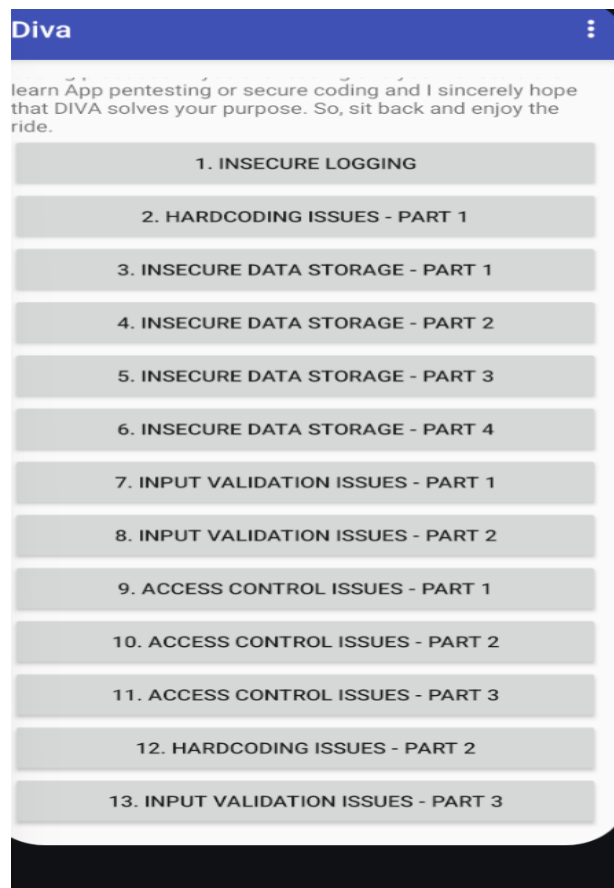


Figure 3 — Full list of DIVA test modules used to structure dynamic testing.

Figures 4–5 — Access Control Bypass (Vendor API Credentials)

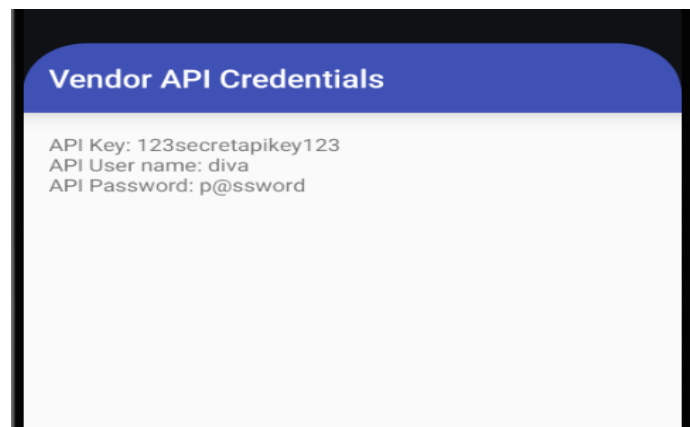


Figure 4 — Vendor API Credentials displayed with no authentication after using Access Control Issues - Part 1.

```
ull packageName=jakhar.aseem.diva fieldId=0 fieldName=null extras=null}, false)
08-30 15:24:57.702 544 2258 I ActivityTaskManager: START u0 {act=jakhar.aseem.diva.action.VIEW_CREDS cmp=jakhar.aseem.diva/.APICredsActivity} with LAUNCH_
MULTIPLE from uid 10190 (BAL_ALLOW_VISIBLE_WINDOW) result code=0
08-30 15:24:57.731 885 920 V WindowManagerShell: Transition requested: android.os.BinderProxy@622e8e5 TransitionRequestInfo { type = OPEN, triggerTask =
TaskInfo{userId=0 taskId=22 displayId=0 isRunning=true baseIntent=Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] flg=0x102000
00 cmp=jakhar.aseem.diva/.MainActivity } baseActivity=ComponentInfo{jakhar.aseem.diva/jakhar.aseem.diva.MainActivity} topActivity=ComponentInfo{jakhar.aseem
diva/jakhar.aseem.diva.APICredsActivity} origActivity=null realActivity=ComponentInfo{jakhar.aseem.diva/jakhar.aseem.diva.MainActivity} numActivities=3 lastA
ctiveTime=5035835 supportsMultiWindow=true resizeMode=4 isResizable=true minWidth=-1 minHeight=-1 defaultMinSize=220 token=WCT{android.window.IWindowContain
```

Figure 5 — logcat confirming the unprotected intent action `jakhar.aseem.diva.action.VIEW_CREDS` launched `APICredsActivity` directly.

Figures 6–8 — Access Control Bypass (“Already Registered”)

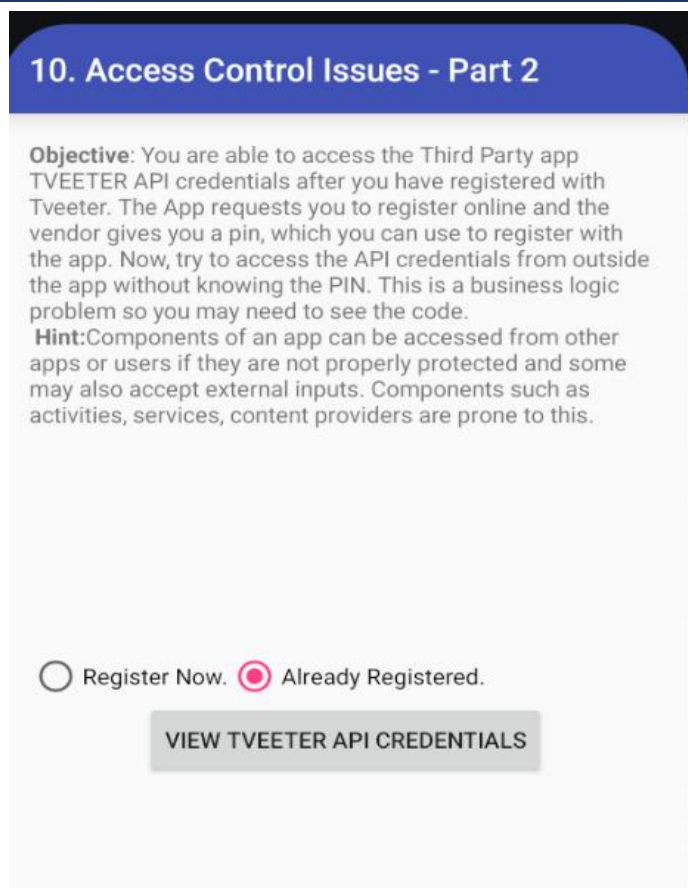


Figure 6 — Access Control Issues - Part 2 screen, showing the “Already Registered” option that skips PIN verification (task objective text visible).

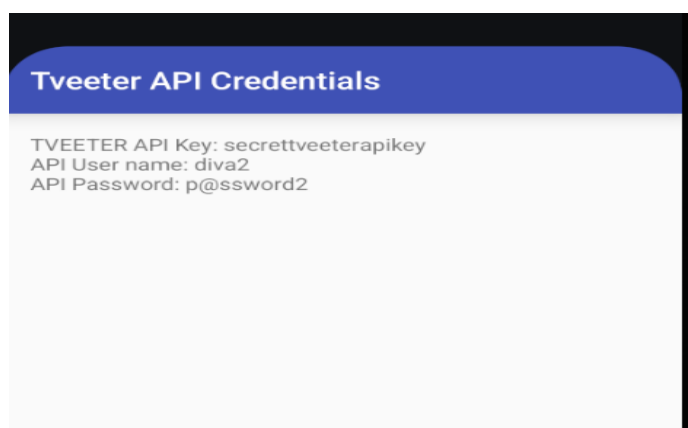


Figure 7 — Tveeter API Credentials displayed immediately after selecting “Already Registered” — no PIN was requested.

```
08-30 15:30:57.309 544 1768 I ActivityTaskManager: START u0 {act=jakhar.aseem.diva.action.VIEW_CREDS2 cmp=jakhar.aseem.diva/.APICreds2Activity (has extras)} with LAUNCH_MULTIPLE from uid 10190 (BAL_ALLOW_VISIBLE_WINDOW) result code=0
08-30 15:30:57.365 885 920 V WindowManagerShell: Transition requested: android.os.BinderProxy@8651b18 TransitionRequestInfo { type = OPEN, triggerTask = TaskInfo{userId=0 taskId=22 displayId=0 isRunning=true baseIntent=Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] flg=0x10200000 cmp=jakhar.aseem.diva/.MainActivity } baseActivity=ComponentInfo{jakhar.aseem.diva/jakhar.aseem.diva.MainActivity} topActivity=ComponentInfo{jakhar.aseem.diva/jakhar.aseem.diva.APICreds2Activity} origActivity=null realActivity=ComponentInfo{jakhar.aseem.diva/jakhar.aseem.diva.MainActivity}}
```

Figure 8 — logcat confirming the second unprotected intent action jakhar.aseem.diva.action.VIEW_CREDS2 launched APICreds2Activity.

Figure 9 — SQL Injection Evidence

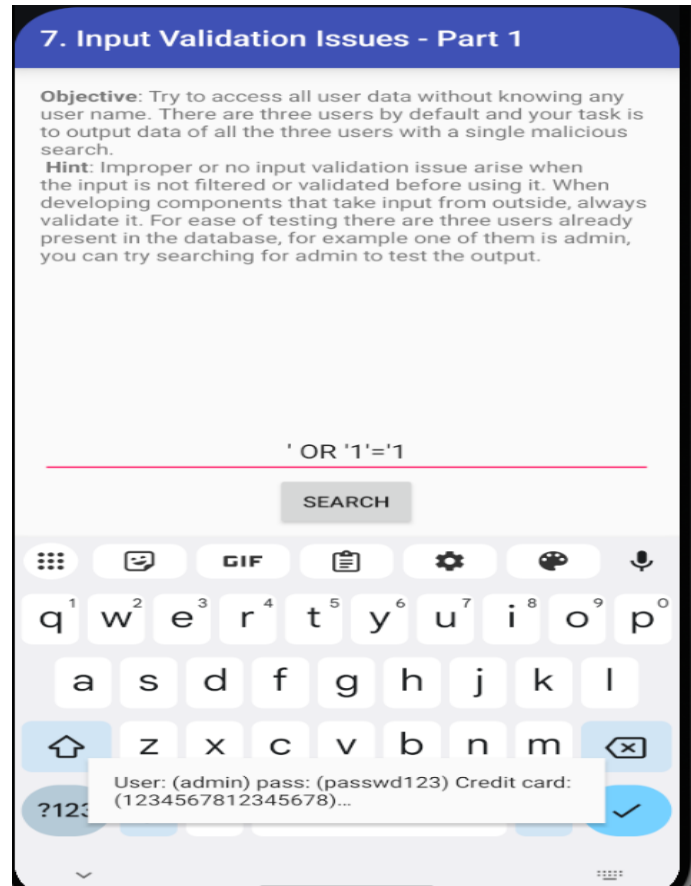


Figure 9 — Input Validation Issues - Part 1: task objective, and the payload ' OR '1'='1 entered in the search field returning admin credentials and a full credit card number in a single query.

Figure 10 — Credentials in External Storage (.uinfo.txt)

```
Administrator: Command Prompt - adb shell
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>adb shell
mu64xa:/ $ cd /sdcard
mu64xa:/sdcard $ ls -la
uinfo.txttotal 112
-rw-rw---- 1 u0_a170 media_rw 21 2026-08-30 15:52 .uinfo.txt
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Alarms
-rwxrws--- 5 media_rw media_rw 4096 2026-08-30 08:09 Android
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Audiobooks
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 DCIM
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Documents
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:34 Download
-rwxrws--- 3 u0_a170 media_rw 4096 2026-08-30 08:09 Movies
-rwxrws--- 3 u0_a170 media_rw 4096 2026-08-30 08:09 Music
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Notifications
-rwxrws--- 3 u0_a170 media_rw 4096 2026-08-30 08:09 Pictures
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Podcasts
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Recordings
-rwxrws--- 2 u0_a170 media_rw 4096 2026-08-30 08:09 Ringtones
mu64xa:/sdcard $ cat .uinfo.txt
testuser:testpass123
mu64xa:/sdcard $
```

Figure 10 — ADB shell listing of /sdcard showing .uinfo.txt, and its contents: testuser:testpass123 in plain text.

Figure 11 — Plaintext SharedPreferences

```

emu64xa:/sdcard $ run-as jakhar.aseem.diva
emu64xa:/data/user/0/jakhar.aseem.diva $ cd shared_prefs
emu64xa:/data/user/0/jakhar.aseem.diva/shared_prefs $ ls
jakhar.aseem.diva_preferences.xml
emu64xa:/data/user/0/jakhar.aseem.diva/shared_prefs $ cat jakhar.aseem.diva_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">testuser</string>
  <string name="notespin">1234</string>
  <string name="user">testuser</string>
</map>
emu64xa:/data/user/0/jakhar.aseem.diva/shared_prefs $

```

Figure 11 — run-as shell access to the app's private shared_prefs file, showing password, user, and notespin (PIN = 1234) stored in plain text.

Figures 12–13 — SQLite Databases (Private Notes and myuser Credentials)

```

C:\Windows\System32>adb exec-out run-as jakhar.aseem.diva cat databases/divanotes.db > divanotes.db
C:\Windows\System32>adb exec-out run-as jakhar.aseem.diva cat databases/ids2 > ids2.db
C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('divanotes.db'); print(c.execute('SELECT name FROM sqlite_master').fetchall())"
[('android_metadata',), ('notes',), ('sqlite_sequence',)]
C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('divanotes.db'); print(c.execute('SELECT * FROM notes').fetchall())"
[(1, 'office', '10 Meetings. 5 Calls. Lunch with CEO'), (2, 'home', 'Buy toys for baby, Order dinner'), (3, 'holiday', 'Either Goa or Amsterdam'), (4, 'Expense', 'Spent too much on home theater'), (5, 'Exercise', 'Alternate days running'), (6, 'Weekend', 'b33333333333r')]
C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('ids2.db'); print(c.execute('SELECT name FROM sqlite_master').fetchall())"
[('android_metadata',), ('myuser',)]

```

Figure 12 — Copying divanotes.db and ids2 to the host machine and listing their tables using Python's sqlite3 module.

```

[('android_metadata',), ('notes',), ('sqlite_sequence',)]

C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('divanotes.db'); print(c.execute('SELECT * FROM notes').fetchall())"
[(1, 'office', '10 Meetings. 5 Calls. Lunch with CEO'), (2, 'home', 'Buy toys for baby, Order dinner'), (3, 'holiday', 'Either Goa or Amsterdam'), (4, 'Expense', 'Spent too much on home theater'), (5, 'Exercise', 'Alternate days running'), (6, 'Weekend', 'b33333333333r')]

C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('ids2.db'); print(c.execute('SELECT name FROM sqlite_master').fetchall())"
[('android_metadata',), ('myuser',)]

C:\Windows\System32>python -c "import sqlite3; c=sqlite3.connect('ids2.db'); print(c.execute('SELECT * FROM myuser').fetchall())"
[('testt', 'testtt'), ('test', 'testt')]

C:\Windows\System32>

```

Figure 13 — Full contents of the notes table (6 private notes in plain text) and the myuser table (2 plaintext username/password pairs).

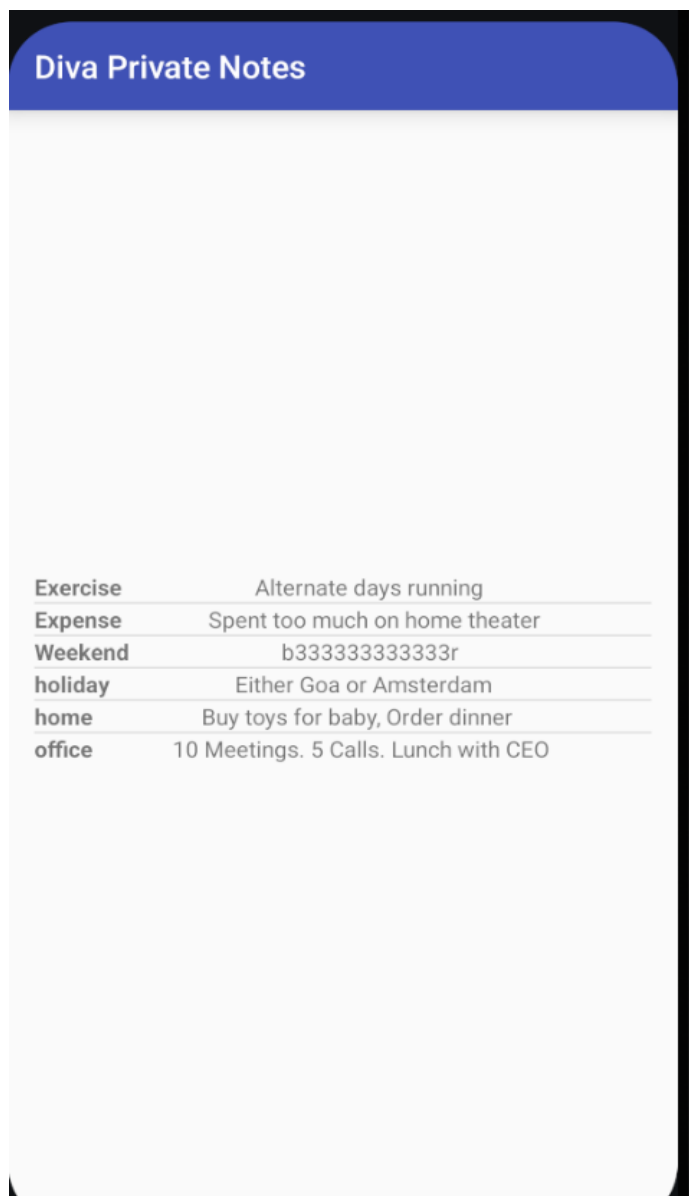
Figure 14 — Private Notes as Displayed in the App

Figure 14 — Diva Private Notes screen, shown after entering the correct PIN through the normal app flow — matching the plaintext data recovered directly from the database in Figure 13.

5. Recommendations

Access Control (Critical Priority — Findings D1, D2)

- Set `android:exported="false"` for `APICredsActivity` and `APICreds2Activity`, and for any other activity that displays sensitive credentials or data, unless external access is genuinely required.
- Never rely on a UI-level flag (such as an “Already Registered” choice) as a substitute for real authentication — verify registration status against a securely stored, tamper-resistant record before granting access.
- Apply the same PIN/authentication enforcement consistently across every activity that displays sensitive data (as correctly done for `AccessControl3NotesActivity` — Finding D8), not only the screen normally reached first through the UI.

SQL Injection (Critical Priority — Finding D3)

- Replace raw, concatenated SQL queries with parameterized queries or prepared statements everywhere user input reaches the database.
- Apply strict input validation and allow-listing on all search and input fields.

Insecure Data Storage (High Priority — Findings D4, D5, D6)

- Never write credentials, PINs, or other authentication secrets to external storage or plain `SharedPreferences`.
- Use the Android Keystore system and encrypted storage APIs (e.g. `EncryptedSharedPreferences`, `SQLCipher` for databases) for any sensitive data that must be stored locally.
- Apply the same encryption-at-rest requirement consistently across every storage mechanism the app uses, since this assessment found the same weakness repeated across files, `SharedPreferences`, and `SQLite`.

Private Data Handling (Medium Priority — Finding D7)

- Encrypt the notes database (or any locally cached personal data) so that PIN protection in the UI is backed by real protection at the storage layer, not just an application-level screen.

General

- Disable debug mode (`android:debuggable="false"`) in production builds, since this setting is what allowed ADB run-as access to the app's private data in Findings D5, D6, and D7.
- Conduct a full access-control review of every exported and non-exported component in the app, since this assessment found the protection level to be inconsistent even within the same feature area.
- Re-test all findings after remediation using the same dynamic (Emulator/Burp/ADB) methodology used in this assessment.

6. Conclusion

Week 6 dynamic testing of DIVA identified 9 findings by running the application on an Android Emulator and observing its live behaviour through network interception (Burp Suite), system logs (logcat), and direct inspection of runtime storage (ADB). Three findings were rated Critical: two independent Access Control bypasses that expose vendor API credentials with no authentication, and a SQL Injection vulnerability that discloses complete user records, including a credit card number, from a single crafted search query.

Three further High-severity findings confirmed, with live and reproducible evidence, that the application stores authentication data in plain text across three separate mechanisms — an external file, SharedPreferences, and a local SQLite database — all of which were readable directly using standard ADB commands. One test returned a safe result (Finding D8), showing that access control is implemented correctly in at least one part of the application, which indicates the vulnerable areas are inconsistencies rather than a complete absence of security awareness in the codebase.

Testing also confirmed, through active monitoring rather than assumption, that the application makes no external network calls, meaning the network-focused checks required by this week's task (insecure API calls, SSL pinning) do not apply here — DIVA's risk is concentrated entirely in on-device access control and data storage. This exercise demonstrated the full dynamic testing lifecycle: environment setup, traffic interception, runtime storage inspection, and evidence-based conclusions, matching the methodology used in real mobile application penetration testing.

----- End of Document -----